**Print copy of official Microsoft Learn documentation**

Source: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_pipelines?view=powershell-7.6
Exported: 2026-10-14. Source metadata date: 2025-12-28T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

# about_Pipelines

## Short description

Combining commands into pipelines in the PowerShell

## Long description

A pipeline is a series of commands connected by pipeline operators (`|`) (ASCII 124). Each pipeline operator sends the results of the preceding command to the next command.

The output of the first command can be sent for processing as input to the second command. And that output can be sent to yet another command. The result is a complex command chain or *pipeline* that's composed of a series of simple commands.

For example,

```
Command-1 | Command-2 | Command-3
```

In this example, the objects that `Command-1` emits are sent to `Command-2`. `Command-2` processes the objects and sends them to `Command-3`. `Command-3` processes the objects and send them down the pipeline. Because there are no more commands in the pipeline, the results are displayed at the console.

In a pipeline, the commands are processed in order from left to right. The processing is handled as a single operation and output is displayed as it's generated.

Here is a simple example. The following command gets the Notepad process and then stops it.

For example,

```
Get-Process notepad | Stop-Process
```

The first command uses the `Get-Process` cmdlet to get an object representing the Notepad process. It uses a pipeline operator (`|`) to send the process object to the `Stop-Process` cmdlet, which stops the Notepad process. Notice that the `Stop-Process` command doesn't have a **Name** or **Id** parameter to specify the process, because the specified process is submitted through the pipeline.

This pipeline example gets the text files in the current directory, selects only the files that are more than 10,000 bytes long, sorts them by length, and displays the name and length of each file in a table.

```
Get-ChildItem -Path *.txt |
  Where-Object {$_.Length -gt 10000} |
    Sort-Object -Property Length |
      Format-Table -Property Name, Length
```

This pipeline consists of four commands in the specified order. The following illustration shows the output from each command as it's passed to the next command in the pipeline.

```
Get-ChildItem -Path *.txt
| (FileInfo objects for *.txt)
V
Where-Object {$_.Length -gt 10000}
| (FileInfo objects for *.txt)
| (      Length > 10000      )
V
Sort-Object -Property Length
| (FileInfo objects for *.txt)
| (      Length > 10000      )
| (     Sorted by length     )
V
Format-Table -Property Name, Length
| (FileInfo objects for *.txt)
| (      Length > 10000      )
| (     Sorted by length     )
| (   Formatted in a table   )
V

Name                        Length
----                        ------
tmp1.txt                     82920
tmp2.txt                    114000
tmp3.txt                    114000
```

## Using pipelines

Most PowerShell cmdlets are designed to support pipelines. In most cases, you can *pipe* the results of a **Get** cmdlet to another cmdlet of the same noun. For example, you can pipe the output of the `Get-Service` cmdlet to the `Start-Service` or `Stop-Service` cmdlets.

This example pipeline starts the WMI service on the computer:

```
Get-Service wmi | Start-Service
```

For another example, you can pipe the output of `Get-Item` or `Get-ChildItem` within the PowerShell Registry provider to the `New-ItemProperty` cmdlet. This example adds a new registry entry, **NoOfEmployees**, with a value of **8124**, to the **MyCompany** registry key.

```
Get-Item -Path HKLM:\Software\MyCompany |
  New-ItemProperty -Name NoOfEmployees -Value 8124
```

Many of the utility cmdlets, such as `Get-Member`, `Where-Object`, `Sort-Object`, `Group-Object`, and `Measure-Object` are used almost exclusively in pipelines. You can pipe any object type to these cmdlets. This example shows how to sort all the processes on the computer by the number of open handles in each process.

```
Get-Process | Sort-Object -Property Handles
```

You can pipe objects to the formatting, export, and output cmdlets, such as `Format-List`, `Format-Table`, `Export-Clixml`, `Export-Csv`, and `Out-File`.

This example shows how to use the `Format-List` cmdlet to display a list of properties for a process object.

```
Get-Process winlogon | Format-List -Property *
```

You can also pipe the output of native commands to PowerShell cmdlets. For example:

```
PS> ipconfig.exe | Select-String -Pattern 'IPv4'

   IPv4 Address. . . . . . . . . . . : 172.24.80.1
   IPv4 Address. . . . . . . . . . . : 192.168.1.45
   IPv4 Address. . . . . . . . . . . : 100.64.108.37
```

> Important
>
> The **Success** and **Error** streams are similar to the stdout and stderr streams of other shells. However, stdin isn't connected to the PowerShell pipeline for input. For more information, see about_Redirection.

With a bit of practice, you'll find that combining simple commands into pipelines saves time and typing, and makes your scripting more efficient.

## How pipelines work

This section explains how input objects are bound to cmdlet parameters and processed during pipeline execution.

### Accepts pipeline input

To support pipelining, the receiving cmdlet must have a parameter that accepts pipeline input. Use the `Get-Help` command with the **Full** or **Parameter** options to determine which parameters of a cmdlet accept pipeline input.

For example, to determine which of the parameters of the `Start-Service` cmdlet accepts pipeline input, type:

```
Get-Help Start-Service -Full
```

or

```
Get-Help Start-Service -Parameter *
```

The help for the `Start-Service` cmdlet shows that only the **InputObject** and **Name** parameters accept pipeline input.

```
-InputObject <ServiceController[]>
Specifies ServiceController objects representing the services to be started.
Enter a variable that contains the objects, or type a command or expression
that gets the objects.

Required?                    true
Position?                    0
Default value                None
Accept pipeline input?       True (ByValue)
Accept wildcard characters?  false

-Name <String[]>
Specifies the service names for the service to be started.

The parameter name is optional. You can use Name or its alias, ServiceName,
or you can omit the parameter name.

Required?                    true
Position?                    0
Default value                None
Accept pipeline input?       True (ByPropertyName, ByValue)
Accept wildcard characters?  false
```

When you send objects through the pipeline to `Start-Service`, PowerShell attempts to associate the objects with the **InputObject** and **Name** parameters.

### Methods of accepting pipeline input

Cmdlets parameters can accept pipeline input in one of two different ways:

- **ByValue**: The parameter accepts values that match the expected .NET type or that can be converted to that type.

  For example, the **Name** parameter of `Start-Service` accepts pipeline input by value. It can accept string objects or objects that can be converted to strings.

- **ByPropertyName**: The parameter accepts input only when the input object has a property of the same name as the parameter.

  For example, the Name parameter of `Start-Service` can accept objects that have a **Name** property. To list the properties of an object, pipe it to `Get-Member`.

Some parameters can accept objects by both value or property name, making it easier to take input from the pipeline.

### Parameter binding

When you pipe objects from one command to another command, PowerShell attempts to associate the piped objects with a parameter of the receiving cmdlet.

PowerShell's parameter binding component associates the input objects with cmdlet parameters according to the following criteria:

- The parameter must accept input from a pipeline.
- The parameter must accept the type of object being sent or a type that can be converted to the expected type.
- The parameter wasn't used in the command.

For example, the `Start-Service` cmdlet has many parameters, but only two of them, **Name** and **InputObject** accept pipeline input. The **Name** parameter takes strings and the **InputObject** parameter takes service objects. Therefore, you can pipe strings, service objects, and objects with properties that can be converted to string or service objects.

PowerShell manages parameter binding as efficiently as possible. You can't suggest or force the PowerShell to bind to a specific parameter. The command fails if PowerShell can't bind the piped objects.

For more information about troubleshooting binding errors, see Investigating Pipeline Errors later in this article.

### One-at-a-time processing

Piping objects to a command is much like using a parameter of the command to submit the objects. Let's look at a pipeline example. In this example, we use a pipeline to display a table of service objects.

```
Get-Service | Format-Table -Property Name, DependentServices
```

Functionally, this is like using the **InputObject** parameter of `Format-Table` to submit the object collection.

For example, we can save the collection of services to a variable that's passed using the **InputObject** parameter.

```
$services = Get-Service
Format-Table -InputObject $services -Property Name, DependentServices
```

Or we can embed the command in the **InputObject** parameter.

```
Format-Table -InputObject (Get-Service) -Property Name, DependentServices
```

However, there's an important difference. When you pipe multiple objects to a command, PowerShell sends the objects to the command one at a time. When you use a command parameter, the objects are sent as a single array object. This minor difference has significant consequences.

When executing a pipeline, PowerShell automatically enumerates any type that implements the `IEnumerable` interface or its generic counterpart. Enumerated items are sent through the pipeline one at a time. PowerShell also enumerates System.Data.DataTable types through the `Rows` property.

There are a few exceptions to automatic enumeration.

- You must call the `GetEnumerator()` method for hash tables, types that implement the `IDictionary` interface or its generic counterpart, and **System.Xml.XmlNode** types.
- The **System.String** class implements `IEnumerable`, however PowerShell doesn't enumerate string objects.

In the following examples, an array and a hashtable are piped to the `Measure-Object` cmdlet to count the number of objects received from the pipeline. The array has multiple members, and the hashtable has multiple key-value pairs. Only the array is enumerated one at a time.

```
@(1,2,3) | Measure-Object
```

```
Count    : 3
Average  :
Sum      :
Maximum  :
Minimum  :
Property :
```

```
@{"One"=1;"Two"=2} | Measure-Object
```

```
Count    : 1
Average  :
Sum      :
Maximum  :
Minimum  :
Property :
```

Similarly, if you pipe multiple process objects from the `Get-Process` cmdlet to the `Get-Member` cmdlet, PowerShell sends each process object, one at a time, to `Get-Member`. `Get-Member` displays the .NET class (type) of the process objects, and their properties and methods.

```
Get-Process | Get-Member
```

```
TypeName: System.Diagnostics.Process

Name      MemberType     Definition
----      ----------     ----------
Handles   AliasProperty  Handles = Handlecount
Name      AliasProperty  Name = ProcessName
NPM       AliasProperty  NPM = NonpagedSystemMemorySize
...
```

Note

`Get-Member` eliminates duplicates, so if the objects are all of the same type, it only displays one object type.

However, if you use the **InputObject** parameter of `Get-Member`, then `Get-Member` receives an array of **System.Diagnostics.Process** objects as a single unit. It displays the properties of an array of objects. (Note the array symbol (`[]`) after the **System.Object** type name.)

For example,

```
Get-Member -InputObject (Get-Process)
```

```
TypeName: System.Object[]

Name                MemberType    Definition
----                ----------    ----------
Count               AliasProperty Count = Length
Address             Method        System.Object& Address(Int32 )
Clone               Method        System.Object Clone()
...
```

This result might not be what you intended. But after you understand it, you can use it. For example, all array objects have a **Count** property. You can use that to count the number of processes running on the computer.

For example,

```
(Get-Process).Count
```

It's important to remember that objects sent down the pipeline are delivered one at a time.

## Using native commands in the pipeline

PowerShell allows you to include native external commands in the pipeline.

Before PowerShell 7.4, piping or redirecting output from a native program that outputs raw byte data converted the output to .NET strings. This conversion caused corruption of the raw data output.

In PowerShell 7.4 or higher, the `PSNativeCommandPreserveBytePipe` experimental feature is mainstream. This feature preserves byte-stream data when redirecting the **stdout** stream of a native command to a file or when piping byte-stream data to the **stdin** stream of a native command.

For example, using the native command `curl` you can download a binary file and save it to disk using redirection.

```
$uri = 'https://github.com/PowerShell/PowerShell/releases/download/v7.3.4/powershell-7.3.4-linux-
arm64.tar.gz'

# native command redirected to a file
curl -s -L $uri > powershell.tar.gz
```

You can also pipe the byte-stream data to the **stdin** stream of another native command. The following example downloads a zipped TAR file using `curl`. The downloaded file data is streamed to the `tar` command to extract the contents of the archive.

```
# native command output piped to a native command
curl -s -L $uri | tar -xzvf - -C .
```

You can also pipe the byte-stream output of a PowerShell command to the input of native command. The following examples use `Invoke-WebRequest` to download the same TAR file as the previous example.

```
# byte stream piped to a native command
(Invoke-WebRequest $uri).Content | tar -xzvf - -C .

# bytes piped to a native command (all at once as byte[])
,(Invoke-WebRequest $uri).Content | tar -xzvf - -C .
```

This feature doesn't support byte-stream data when redirecting **stderr** output to **stdout**. When you combine the **stderr** and **stdout** streams, the combined streams are treated as string data.

## Investigating pipeline errors

When PowerShell can't associate the piped objects with a parameter of the receiving cmdlet, the command fails.

In the following example, we try to move a registry entry from one registry key to another. The `Get-Item` cmdlet gets the destination path, which is then piped to the `Move-ItemProperty` cmdlet. The `Move-ItemProperty` command specifies the current path and name of the registry entry to be moved.

```
Get-Item -Path HKLM:\Software\MyCompany\sales |
Move-ItemProperty -Path HKLM:\Software\MyCompany\design -Name product
```

The command fails and PowerShell displays the following error message:

```
Move-ItemProperty : The input object can't be bound to any parameters for
the command either because the command doesn't take pipeline input or the
input and its properties do not match any of the parameters that take
pipeline input.
At line:1 char:23
+ $a | Move-ItemProperty <<<<  -Path HKLM:\Software\MyCompany\design -Name p
```

To investigate, use the `Trace-Command` cmdlet to trace the parameter binding component of PowerShell. The following example traces parameter binding while the pipeline is executing. The **PSHost** parameter displays the trace results in the console and the **FilePath** parameter send the trace results to the `debug.txt` file for later reference.

```
Trace-Command -Name ParameterBinding -PSHost -FilePath debug.txt -Expression {
  Get-Item -Path HKLM:\Software\MyCompany\sales |
    Move-ItemProperty -Path HKLM:\Software\MyCompany\design -Name product
}
```

The results of the trace are lengthy, but they show the values being bound to the `Get-Item` cmdlet and then the named values being bound to the `Move-ItemProperty` cmdlet.

```
...
BIND NAMED cmd line args [`Move-ItemProperty`]
BIND arg [HKLM:\Software\MyCompany\design] to parameter [Path]
...
BIND arg [product] to parameter [Name]
...
BIND POSITIONAL cmd line args [`Move-ItemProperty`]
...
```

Finally, it shows that the attempt to bind the path to the **Destination** parameter of `Move-ItemProperty` failed.

```
...
BIND PIPELINE object to parameters: [`Move-ItemProperty`]
PIPELINE object TYPE = [Microsoft.Win32.RegistryKey]
RESTORING pipeline parameter's original values
Parameter [Destination] PIPELINE INPUT ValueFromPipelineByPropertyName NO
COERCION
Parameter [Credential] PIPELINE INPUT ValueFromPipelineByPropertyName NO
COERCION
...
```

Use the `Get-Help` cmdlet to view the attributes of the **Destination** parameter.

```
Get-Help Move-ItemProperty -Parameter Destination

-Destination <String>
    Specifies the path to the destination location.

    Required?                    true
    Position?                    1
    Default value                None
    Accept pipeline input?       True (ByPropertyName)
    Accept wildcard characters?  false
```

The results show that **Destination** takes pipeline input only "by property name". Therefore, the piped object must have a property named **Destination**.

Use `Get-Member` to see the properties of the object coming from `Get-Item`.

```
Get-Item -Path HKLM:\Software\MyCompany\sales | Get-Member
```

The output shows that the item is a **Microsoft.Win32.RegistryKey** object that doesn't have a **Destination** property. That explains why the command failed.

The **Path** parameter accepts pipeline input by name or by value.

```
Get-Help Move-ItemProperty -Parameter Path

-Path <String[]>
    Specifies the path to the current location of the property. Wildcard
    characters are permitted.
```

```
    Required?                    true
    Position?                    0
    Default value                None
    Accept pipeline input?       True (ByPropertyName, ByValue)
    Accept wildcard characters?  true
```

To fix the command, we must specify the destination in the `Move-ItemProperty` cmdlet and use `Get-Item` to get the **Path** of the item we want to move.

For example,

```
Get-Item -Path HKLM:\Software\MyCompany\design |
Move-ItemProperty -Destination HKLM:\Software\MyCompany\sales -Name product
```

## Intrinsic line continuation

As already discussed, a pipeline is a series of commands connected by pipeline operators (`|`), usually written on a single line. However, for readability, PowerShell allows you to split the pipeline across multiple lines. When a pipe operator is the last token on the line, the PowerShell parser joins the next line to current command to continue the construction of the pipeline.

For example, the following single-line pipeline:

```
Command-1 | Command-2 | Command-3
```

can be written as:

```
Command-1 |
    Command-2 |
    Command-3
```

The leading spaces on the subsequent lines aren't significant. The indentation enhances readability.

PowerShell 7 adds support for continuation of pipelines with the pipeline character at the beginning of a line. The following examples show how you could use this new functionality.

```
# Wrapping with a pipe at the beginning of a line (no backtick required)
Get-Process | Where-Object CPU | Where-Object Path
    | Get-Item | Where-Object FullName -Match "AppData"
    | Sort-Object FullName -Unique

# Wrapping with a pipe on a line by itself
Get-Process | Where-Object CPU | Where-Object Path
    |
    Get-Item | Where-Object FullName -Match "AppData"
    |
    Sort-Object FullName -Unique
```

> Important

When working interactively in the shell, pasting code with pipelines at the beginning of a line only when using `Ctrl+V` to paste. Right-click paste operations insert the lines one at a time. Since the line doesn't end with a pipeline character, PowerShell considers the input to be complete and executes that line as entered.

## See also

- about_Objects
- about_Parameters
- about_Command_Syntax
- about_Foreach